고객을 세그먼테이션하자 [프로젝트] - 김경국

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT *
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
LIMIT 10
```

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIGH...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT ...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*) AS total_rows
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	total_rows
1	541909

데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT
  COUNT(InvoiceNo) AS COUNT_InvoiceNo,
  COUNT(StockCode) AS COUNT_StockCode,
  COUNT(Description) AS COUNT_Description,
  COUNT(Quantity) AS COUNT_Quantity,
  COUNT(InvoiceDate) AS COUNT_InvoiceDate,
  COUNT(UnitPrice) AS COUNT_UnitPrice,
  COUNT(CustomerID) AS COUNT_CustomerID,
  COUNT(Country) AS COUNT_Country
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Descripti...	COUNT_Quantity	COUNT_InvoiceD...	COUNT_UnitPrice	COUNT_Custome...	COUNT_Country
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT
  'Description' AS COUNT_Description,
  ROUND((COUNT(*) - COUNT(Description)) / COUNT(*) * 100, 2) AS missing_ratio
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
UNION ALL
SELECT
  'CustomerID' AS COUNT_Description,
  ROUND((COUNT(*) - COUNT(CustomerID)) / COUNT(*) * 100, 2) AS missing_ratio
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	COUNT_Description	missing_ratio
1	Description	0.27
2	CustomerID	24.93

결측치 처리 전략

- `StockCode = '85123A'` **의** `Description` **을 추출하는 쿼리문을 작성하기**

```
SELECT DISTINCT Description
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
WHERE StockCode = '85123A'
```

행	Description
1	WHITE HANGING HEART T-LIGHT HOLDER
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIGHT HOLDER

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
DELETE FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
WHERE CustomerID IS NULL
OR Description = '?'
```

이 문으로 기본 테이블의 행 135,080개가 삭제되었습니다.

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
SELECT COUNT(*) AS duplicate_cnt
FROM (
  SELECT *
```

```
  FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
  GROUP BY
    InvoiceNo,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
    Country
  HAVING COUNT(*) > 1
)
```

행	duplicate_cnt
1	4837

중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블` AS
SELECT DISTINCT *
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

이 문으로 이름이 기본 테이블인 테이블이 교체되었습니다.

남은 데이터 수

행	f0_
1	401604

11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo`의 개수를 출력하기**

```
SELECT COUNT(DISTINCT InvoiceNo) AS Invoice_count
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	Invoice_count
1	22190

- **고유한 `InvoiceNo`를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
LIMIT 100
```

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318
11	541998
12	548955

- `InvoiceNo` 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100
```

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC
5	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC
6	C547388	22645	CERAMIC HEART FAIRY CAKE M...	-12	2011-03-22 16:07:00 UTC
7	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC
8	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC
9	C547388	37448	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC
10	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC
11	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC
12	C549955	22839	3 TIER CAKE TIN GREEN AND C...	-2	2011-04-13 13:38:00 UTC

- 구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT ROUND((SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END)/ COUNT(*) * 100), 1) AS Cancele
d_Ratio
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	Canceled_Ratio
1	2.2

`StockCode` 살펴보기

- 고유한 `StockCode` 의 개수를 출력하기

```
SELECT COUNT(DISTINCT StockCode) AS StockCode_cnt
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	StockCode_cnt
1	3684

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10
```

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196

- **`StockCode` 의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
)
WHERE number_count BETWEEN 0 AND 1
```

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- **`StockCode` 의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```
WITH letter AS (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode,
      LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
  )
  WHERE number_count BETWEEN 0 AND 1
)
SELECT
ROUND(
  COUNT(CASE WHEN StockCode IN (SELECT StockCode FROM letter) THEN 1 END)
  / COUNT(*) * 100
, 2) AS ratio_percent
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`;
```

행	ratio_percent
1	0.48

- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블
WHERE StockCode IN (
SELECT DISTINCT StockCode
FROM (
SELECT StockCode,
LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
FROM project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블
)
WHERE number_count BETWEEN 0 AND 1
)
```

이 문으로 기본 테이블의 행 1,915개가 삭제되었습니다.

`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
GROUP BY Description
LIMIT 30
```

행	Description	description_cnt
1	MEDIUM CERAMIC TOP STORA...	208
2	SET OF 2 TINS VINTAGE BATHR...	59
3	ALARM CLOCK BAKELIKE PINK	636
4	ALARM CLOCK BAKELIKE GREEN	819
5	3D DOG PICTURE PLAYING CAR...	61
6	MINI PAINT SET VINTAGE	335
7	PURPLE DRAWERKNOB ACRYLI...	151
8	BLUE DRAWER KNOB ACRYLIC ...	124

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
WHERE Description IN ('Next Day Carriage', 'High Resolution Image')
```

이 문으로 기본 테이블의 행 83개가 삭제되었습니다.

- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

이 문으로 이름이 기본 테이블인 테이블이 교체되었습니다.

`UnitPrice` 살펴보기

- **`UnitPrice`의 최솟값, 최댓값, 평균을 구하기**

```
SELECT MIN(UnitPrice) AS min_price, MAX(UnitPrice) AS max_price, AVG(UnitPrice) AS avg_price
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	min_price	max_price	avg_price
1	0.0	649.5	2.904956757406...

- **단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기**

```
SELECT
  COUNT(*) AS cnt_quantity,
  MIN(Quantity) AS min_quantity,
```

```
  MAX(Quantity) AS max_quantity,
  AVG(Quantity) AS avg_quantity
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
WHERE UnitPrice = 0
```

행	cnt_quantity	min_quantity	max_quantity	avg_quantity
1	33	1	12540	420.5151515151...

- `UnitPrice = 0` **를 제거하고 일관된 데이터셋을 유지하기**

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블` AS
SELECT *
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
WHERE UnitPrice != 0
```

이 문으로 이름이 기본 테이블인 테이블이 교체되었습니다.

11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT DATE(InvoiceDate) AS InvoiceDay, *
FROM project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블
```

행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC
3	2010-12-07	537626	20782	6	2010-12-07 14:57:00 UTC
4	2010-12-07	537626	22375	4	2010-12-07 14:57:00 UTC
5	2010-12-07	537626	71477	12	2010-12-07 14:57:00 UTC
6	2010-12-07	537626	22497	4	2010-12-07 14:57:00 UTC
7	2010-12-07	537626	22775	12	2010-12-07 14:57:00 UTC

- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
SELECT
  DATE(MAX(InvoiceDate) OVER()) AS most_recent_date,
  DATE(InvoiceDate) AS InvoiceDay,
  *
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
```

행	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Quantity
1	2011-12-09	2011-01-18	541431	23166	74215
2	2011-12-09	2011-01-18	C541433	23166	-74215
3	2011-12-09	2010-12-07	537626	20782	6
4	2011-12-09	2010-12-07	537626	22375	4
5	2011-12-09	2010-12-07	537626	71477	12
6	2011-12-09	2010-12-07	537626	22497	4
7	2011-12-09	2010-12-07	537626	22775	12

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
  CustomerID,
  MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
GROUP BY CustomerID
```

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21

- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM project_name.modulabs_project.data
  GROUP BY CustomerID
);
```

행	CustomerID	recency
1	12559	310
2	12738	372
3	12842	70
4	13334	82
5	13350	16
6	13355	122
7	13560	7
8	13579	14

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_r` AS
WITH most_recent_date AS (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
  GROUP BY CustomerID
)
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER() - InvoiceDay) AS recency
FROM most_recent_date
```

행	CustomerID	recency
1	17001	0
2	12985	0
3	16954	0
4	12662	0
5	17315	0
6	13426	0
7	12680	0
8	17581	0
9	12713	0
10	14397	0
11	15311	0
12	17389	0
13	14422	0
14	16446	0
15	14051	0
16	17428	0
17	15910	0
18	12423	0

Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
GROUP BY CustomerID
```

행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1

- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
GROUP BY CustomerID
```

행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530

- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_rf` AS
WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
  FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
  GROUP BY CustomerID
),
```

```
item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
  GROUP BY CustomerID
)

SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_r` AS ur
  ON pc.CustomerID = ur.CustomerID;
```

행	CustomerID	purchase_cnt	item_cnt	recency
1	12713	1	505	0
2	14569	1	79	1
3	13436	1	76	1
4	13298	1	96	1
5	15520	1	314	1
6	15471	1	256	2
7	15195	1	1404	2
8	14204	1	72	2
9	15318	1	642	3
10	17914	1	457	3
11	12650	1	250	3
12	16528	1	171	3
13	14578	1	240	3
14	15992	1	17	3
15	12442	1	181	3
16	12478	1	233	3
17	16569	1	93	3
18	17383	1	148	4

Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
CustomerID,
ROUND(SUM(UnitPrice * Quantity)) AS user_total
FROM project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블
GROUP BY CustomerID
```

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.0
4	12349	1458.0
5	12350	294.0
6	12352	1265.0
7	12353	89.0
8	12354	1079.0

- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ut. user_total /SUM(user_total) AS user_average
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_rf` rf
LEFT JOIN (
  SELECT
  CustomerID,
  ROUND(SUM(UnitPrice * Quantity)) AS user_total
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
GROUP BY CustomerID

) ut
ON rf.CustomerID = ut.CustomerID;
```

이 문으로 이름이 user_rfm인 새 테이블이 생성되었습니다.

RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_rfm`
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	795.0	795.0
2	13298	1	96	1	360.0	360.0
3	15520	1	314	1	343.0	343.0
4	14569	1	79	1	227.0	227.0
5	13436	1	76	1	197.0	197.0
6	15195	1	1404	2	3861.0	3861.0
7	15471	1	256	2	454.0	454.0

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
  GROUP BY CustomerID
)
SELECT
  ur.*,
  up.* EXCEPT (CustomerID)
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
  ON ur.CustomerID = up.CustomerID;
```

이 문으로 이름이 user_data인 새 테이블이 생성되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...
1	12713	1	505	0	795.0	795.0	37
2	13436	1	76	1	197.0	197.0	12
3	15520	1	314	1	343.0	343.0	18
4	14569	1	79	1	227.0	227.0	10
5	13298	1	96	1	360.0	360.0	2
6	14204	1	72	2	151.0	151.0	36
7	15195	1	1404	2	3861.0	3861.0	1
8	15471	1	256	2	454.0	454.0	67
9	12442	1	181	3	144.0	144.0	11
10	15992	1	17	3	42.0	42.0	3
11	14578	1	240	3	169.0	169.0	24
12	12650	1	250	3	242.0	242.0	19
13	16528	1	171	3	244.0	244.0	17
14	15318	1	642	3	313.0	313.0	33
15	12478	1	233	3	546.0	546.0	35
16	17914	1	457	3	329.0	329.0	72
17	16569	1	93	3	124.0	124.0	5
18	13790	1	748	4	349.0	349.0	45

2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **평균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합**

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_data` AS
WITH purchase_intervals AS (
  SELECT
    CustomerID,
    CASE
      WHEN ROUND(AVG(interval_),2) IS NULL THEN 0
      ELSE ROUND(AVG(interval_),2)
    END AS average_interval
  FROM (
    SELECT
      CustomerID,
      DATE_DIFF(
        DATE(InvoiceDate),
        LAG(DATE(InvoiceDate)) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate),
        DAY
      ) AS interval_
    FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)

SELECT
  u.*,
  pi.average_interval
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inter...
1	17986	1	10	56	21.0	21.0	1	0.0
2	16148	1	72	296	76.0	76.0	1	0.0
3	13270	1	200	366	590.0	590.0	1	0.0
4	15313	1	25	110	52.0	52.0	1	0.0
5	13185	1	12	267	71.0	71.0	1	0.0
6	18133	1	1350	212	931.0	931.0	1	0.0
7	16257	1	1	176	22.0	22.0	1	0.0
8	16765	1	4	294	34.0	34.0	1	0.0
9	17752	1	192	359	81.0	81.0	1	0.0
10	16881	1	600	66	432.0	432.0	1	0.0
11	16138	1	-1	368	-8.0	-8.0	1	0.0
12	17102	1	2	261	26.0	26.0	1	0.0
13	15316	1	100	326	165.0	165.0	1	0.0
14	18233	1	4	325	440.0	440.0	1	0.0
15	15070	1	36	372	106.0	106.0	1	0.0
16	16344	1	18	158	101.0	101.0	1	0.0
17	16454	1	2	64	6.0	6.0	1	0.0
18	13307	1	4	120	15.0	15.0	1	0.0

3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
    (취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_data` AS

WITH TransactionInfo AS (
  SELECT
```

```
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency
  FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.기본 테이블`
  GROUP BY CustomerID
)

SELECT
  u.*,
  t.* EXCEPT(CustomerID),
  ROUND(t.cancel_frequency / t.total_transactions, 2) AS cancel_rate
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inter...	total_transac...	cancel_frequ...	cancel_rate
1	15286	1	55	49	241.0	241.0	3	0.0	1	0	0.0
2	12789	1	3	65	77.0	77.0	3	0.0	1	0	0.0
3	15368	1	230	21	447.0	447.0	4	0.0	1	0	0.0
4	15347	1	130	366	202.0	202.0	5	0.0	1	0	0.0
5	16190	1	88	40	215.0	215.0	6	0.0	1	0	0.0
6	16828	1	58	112	114.0	114.0	7	0.0	1	0	0.0
7	13161	1	29	313	113.0	113.0	8	0.0	1	0	0.0
8	17165	1	100	284	159.0	159.0	10	0.0	1	0	0.0
9	15935	1	114	239	108.0	108.0	11	0.0	1	0	0.0
10	15925	1	93	194	207.0	207.0	12	0.0	1	0	0.0
11	15692	1	138	28	191.0	191.0	12	0.0	1	0	0.0
12	16125	1	124	367	289.0	289.0	13	0.0	1	0	0.0
13	17893	1	149	225	107.0	107.0	15	0.0	1	0	0.0
14	13512	1	143	260	251.0	251.0	15	0.0	1	0	0.0
15	18251	1	7824	87	4315.0	4315.0	16	0.0	1	0	0.0
16	15885	1	109	93	349.0	349.0	16	0.0	1	0	0.0
17	15520	1	314	1	343.0	343.0	18	0.0	1	0	0.0
18	13304	1	238	351	300.0	300.0	21	0.0	1	0	0.0

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
SELECT *
FROM `project-620ca1cc-54cc-4306-8d4.modulabs_project.user_data`
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_prod...	average_inter...	total_transac...	cancel_frequ...	cancel_rate
1	15286	1	55	49	241.0	241.0	3	0.0	1	0	0.0
2	12789	1	3	65	77.0	77.0	3	0.0	1	0	0.0
3	15368	1	230	21	447.0	447.0	4	0.0	1	0	0.0
4	15347	1	130	366	202.0	202.0	5	0.0	1	0	0.0
5	16190	1	88	40	215.0	215.0	6	0.0	1	0	0.0
6	16828	1	58	112	114.0	114.0	7	0.0	1	0	0.0
7	13161	1	29	313	113.0	113.0	8	0.0	1	0	0.0
8	17165	1	100	284	159.0	159.0	10	0.0	1	0	0.0
9	15935	1	114	239	108.0	108.0	11	0.0	1	0	0.0
10	15925	1	93	194	207.0	207.0	12	0.0	1	0	0.0
11	15692	1	138	28	191.0	191.0	12	0.0	1	0	0.0
12	16125	1	124	367	289.0	289.0	13	0.0	1	0	0.0
13	17893	1	149	225	107.0	107.0	15	0.0	1	0	0.0
14	13512	1	143	260	251.0	251.0	15	0.0	1	0	0.0
15	18251	1	7824	87	4315.0	4315.0	16	0.0	1	0	0.0
16	15885	1	109	93	349.0	349.0	16	0.0	1	0	0.0
17	15520	1	314	1	343.0	343.0	18	0.0	1	0	0.0
18	13304	1	238	351	300.0	300.0	21	0.0	1	0	0.0

회고

[회고 내용을 작성해주세요]

Keep : LMS 학습 방향이 잡혀있어서 따라가는데 어렵지 않았다.

Problem :

1. SQL 문법적 오류 자주 발생함 EX) GROUP BY 없이 COUNT 사용
2. 집계 함수랑 일반 컬럼을 같이 사용할 때 헷갈렸다.
3. 이전 데이터가 덮어써져서 duplicate column이 자주 발생
4. 실습을 진행하면서 LMS에서 제시된 방향을 따라가는 데 집중하다 보니, 중간에 떠올랐던 다른 분석 방법이나 접근 방식을 충분히 시도해보지 못했다. 시간에 쫓겨 주어진 절차를 그대로 수행하는 데 그친 점이 아쉬웠다.

Try :

1. 데이터 구조를 더 제대로 파악
2. 집계 함수나 지표의 의미를 생각해보기
3. 앞으로 다른 데이터를 분석할 때는 시간이 조금 더 걸리더라도, 주어진 가이드라인만 따르기보다 스스로 생각한 방법이나 추가적인 분석도 함께 시도해보고 싶다.